

基于 SIMD 的 ANN 多线程与加速器并行优化

Pthread、HNSW、OpenMP Target 与跨平台性能分析

2026 年 5 月 20 日

姓名：匡航逸 学号：2412235
专业：计算机科学与技术 数据集：DEEP100K
评价指标：recall@100，并在 recall@100 $\geq$ 0.95 下最小化平均查询延迟
GitHub: <https://github.com/xzxxntxdy/Parallel-programming-NKU>

摘要

本报告在 SIMD 阶段 Flat、SQ8、PQ、SDC 与 FastScan 距离计算优化的基础上，继续研究近似最近邻搜索（Approximate Nearest Neighbor Search, ANNS）的多线程与加速器并行化。实验统一采用 DEEP100K 数据集，并在 recall@100 $\geq$ 0.95 的候选中选择最低查询延迟。围绕这一目标，本文依次实现并分析了 Flat 精确搜索的 Pthread 并行、SQ8 与 PQ 的候选压缩、SDC/FastScan 的查表优化、IVF/IVF-PQ 的倒排裁剪、HNSW 图索引及其查询内并行变体，以及 Intel GPU 上的 OpenMP target batch IVF 卸载。x86 平台最终最优为 HNSW query-level 并行：StdAsync, ef=64, threads=32，全量查询平均延迟 9.316 μ s/query, recall@100=0.952683；ARM/Kunpeng 平台最终最优为 StdThread, ef=64, threads=16，全量 test.sh 结果为 70.8786 μ s/query, recall@100=0.952643。OpenMP target 在 Intel GPU 上严格执行设备卸载，最优 IVF 配置为 nprobe=81, candidate cap=3840，延迟 0.186502 ms/query, recall@100=0.950170。实验表明：多线程的收益取决于算法瓶颈，HNSW 的图结构剪枝远比简单扩大线程数有效；GPU 路径必须结合候选裁剪和批处理，才能把设备吞吐转化为端到端收益。

目录

1 问题定义与评价指标	4
1.1 kNN 与 ANNS 问题	4
1.2 recall@100 与最优选择规则	4
1.3 实验平台	4
2 实验矩阵与消融设计	5
3 并行开销模型	6
4 总体优化路线	6
5 Flat-SIMD 与 Pthread 并行	8

5.1	算法设计	8
5.2	优化过程与实验结果	8
6	SQ8、PQ、SDC 与 FastScan	9
6.1	SQ8: 标量量化与 rerank	9
6.2	PQ-ADC 与 PQ-SDC	9
6.3	FastScan 与查表布局	10
6.4	量化路径的优化复盘	11
7	IVF 与 IVF-PQ	12
7.1	倒排索引结构	12
7.2	nprobe 的 trade-off	13
7.3	IVF-PQ 与局部 PQ	14
8	HNSW 图索引	14
8.1	HNSW 查询过程	14
8.2	query-level 并行	14
8.3	查询内并行探索	15
9	OpenMP Target 与 oneAPI/SYCL	17
9.1	加速器实现目标	17
9.2	OpenMP target IVF	17
9.3	与 oneAPI/SYCL 的比较	18
10	x86 与 ARM/Kunpeng 跨平台分析	20
10.1	算法最优点是否迁移	20
10.2	构建时间与查询延迟	20
10.3	最终参数选择依据	21
11	综合讨论：算法结构与硬件匹配	21
11.1	从计算密集到访存密集	21
11.2	线程粒度的选择原则	22
11.3	近似误差与参数敏感性	22
11.4	跨平台差异的解释	23
11.5	对最终方案的工程判断	23
12	优化过程总结	23
13	结论	23
A	完整图表与复现说明	24
B	生成式 AI 辅助学习附录	24

1 问题定义与评价指标

1.1 kNN 与 ANNS 问题

给定 base 集合 $X = \{x_i\}_{i=1}^N$ 、查询向量 q 和距离函数 $d(\cdot, \cdot)$ ，精确 k 近邻搜索要求返回距离最小的 k 个向量：

$$\text{kNN}(q) = \arg \min_{S \subset X, |S|=k} \sum_{x \in S} d(q, x).$$

在 DEEP100K 上， $N = 100000$ ，向量维度 $d = 96$ ，ground truth 提供 top-100。因此本文的目标输出为每个 query 的 top-100 候选。精确 Flat 搜索需要访问全部 base 向量，计算复杂度为 $O(Nd)$ ；ANNS 则通过量化、倒排索引或图索引减少候选访问量，以可控的 recall 换取更低 latency。

本文使用的距离形式与 SIMD 阶段一致。每个候选向量与 query 的相似度或距离由 SIMD 核进行批量计算。Pthread 阶段的新增问题是：当单次距离计算已经被 SIMD 压缩之后，继续增加线程是否仍然有效？如果有效，线程应该并行 query、base、倒排链、PQ 查表，还是图搜索入口？这些问题决定了后续算法设计。

1.2 recall@100 与最优选择规则

设算法返回集合为 R_q ，ground truth 集合为 G_q ，两者大小均为 100，则单个 query 的召回率为

$$\text{recall@100}(q) = \frac{|R_q \cap G_q|}{100}.$$

实验报告中的 recall 为所有 query 的平均值。本文采用如下约束优化准则：

$$\theta^* = \arg \min_{\theta \in \Theta} \text{Latency}(\theta) \quad \text{s.t.} \quad \text{recall@100}(\theta) \geq 0.95.$$

其中 θ 包括算法、线程数、候选数 p 、倒排簇数 nlist 、搜索簇数 nprobe 、HNSW 的 ef 、GPU candidate cap 等参数。后续所有表格和图中的“最优”均按这一规则筛选。

1.3 实验平台

实验覆盖三个层次：x86 Windows 本地 CPU、ARM/Kunpeng CPU，以及 x86 机器上的 Intel GPU 加速器。x86 CPU sweep 使用 1000 个 query 进行完整消融；ARM/Kunpeng 使用 quick sweep 选择参数，并通过全量 `test.sh` 验证最终路径；GPU 实验采用 `batch=1000` 的 IVF 设备端扫描。

表 1: 实验平台与评价口径

平台	硬件与后端	运行规模	作用
x86 Windows	Intel Core i9-14900HX, AVX2, MSVC / Intel DPC++ 2024.2.1	CPU sweep 1000 queries; final 全量 queries	本地完整消融、HNSW、OpenMP target 与 SYCL 对比。
ARM/Kunpeng	HiSilicon Kunpeng-920, aarch64, NEON, OpenEuler 5.10	quick sweep 500 queries; final 全量 queries	跨平台分析与 ARM 最终提交路径。
Intel GPU	Intel RaptorLake-S Mobile Graphics Controller, Level Zero / OpenCL	batch=1000	OpenMP target 与 oneAPI/SYCL 加速器实验。

2 实验矩阵与消融设计

ANN 多线程优化容易出现“单点最优不可解释”的问题：某个配置可能在一次运行中很快，但无法说明它是算法结构收益、线程调度收益，还是 recall 降低带来的假象。本文因此采用分层消融矩阵。第一层固定算法族，扫描线程数，观察并行扩展；第二层固定线程数，扫描候选规模或搜索宽度，观察 recall-latency 曲线；第三层比较不同平台上的最优点，判断参数是否具有迁移性。

实验矩阵不是为了穷举所有可能参数，而是围绕每类算法的关键控制量展开。Flat 的关键量是线程数和局部 top- p ；SQ8/PQ/FastScan 的关键量是量化粒度、候选数 p 和查表布局；IVF 的关键量是 `nlist` 与 `nprobe`；HNSW 的关键量是 `ef`、线程工具和查询内并行策略；GPU IVF 的关键量是 `batch`、`nprobe`、`candidate cap`、距离矩阵回传规模与 top-k selection 开销。

从任务要求看，本文的实验覆盖了三个层次。第一，CPU 端以 Pthread-style 多线程为主线，所有 Flat、量化、IVF 和 HNSW 路径都显式记录线程数、线程划分方式和局部结果归并策略。第二，图索引进阶部分不只运行标准 HNSW，还实现了边划分、Layer 0 点划分、多入口点并行和 IVF+HNSW 嵌套结构，并如实比较这些查询内并行策略的正负收益。第三，OpenMP 部分严格采用 target device 卸载到 Intel GPU，而不是把普通 CPU OpenMP 当作完成项；oneAPI/SYCL 作为异构编程工具对比，统一用同一数据集、同一 $\text{recall}@100 \geq 0.95$ 和同一 CSV 汇总脚本评价。ARM/Kunpeng 平台由于本地不可直接运行，在代码和命令上保留 quick sweep 与 final 验证入口，实测结果回填到同一报告体系中。

表 2: 主要消融维度与预期观察

算法族	消融参数	主要观测指标	预期 trade-off
Flat	SIMD kernel、线程数、local top- p	latency、recall、线程扩展	精确但扫描全量；base-split 需要 top-k merge。
SQ8	候选数 p 、rerank 规模、线程数	近似筛选时间、精排时间	p 增大提升 recall，但 rerank 抬高延迟。
PQ/SDC	子空间数 M 、候选数 p 、ADC/SDC	LUT 构建、查表累加、recall	查表降低计算量，但随机访存更明显。
FastScan	block 大小、候选数 p 、线程数	SIMD 查表吞吐、cache 行为	layout 越适合 SIMD，单候选越快；候选仍不能过多。
IVF	<code>nlist</code> 、 <code>nprobe</code> 、线程数	coarse 时间、scan 时间、recall	<code>nprobe</code> 是 recall 与扫描量的直接旋钮。
HNSW	<code>ef</code> 、线程数、查询内并行策略	graph visit 数、队列维护、recall	query-level 稳定；查询内切分可能破坏剪枝。
OpenMP target	<code>nprobe</code> 、 <code>candidate cap</code> 、 <code>batch</code>	GPU scan、selection、传输/同步	需要足够 batch 和候选裁剪，selection 下沉与回传规模是后续关键。

所有消融结果都保留构建时间和查询时间的区分。构建时间不进入单 query latency，因为课程评测关注查询性能；但构建代价会影响算法选择。例如 Flat 没有构建时间，但查询慢；HNSW 构建时间更高，但查询显著更快。因此本文在最终分析中同时报告 build time 和 query latency。

为了避免 recall 更低的配置被误判为“优化成功”，每个实验族都先过滤 $\text{recall}@100 \geq 0.95$ ，再按 latency 排序。没有达到 0.95 的点仍然出现在 trade-off 图中，但不会进入最优表。HNSW 查询内并行、

IVF-HNSW 和 GPU batch IVF 都按同一规则评价，保证不同并行工具之间的比较口径一致。

3 并行开销模型

多线程优化的实际收益可以用一个简化模型理解。对某个算法配置，平均查询延迟可以拆成

$$T(P) = \frac{T_{\text{work}}}{S(P)} + T_{\text{sync}}(P) + T_{\text{merge}}(P) + T_{\text{mem}}(P),$$

其中 P 为线程数， $S(P)$ 是有效并行加速比。理想情况下 $S(P) \approx P$ ，但 ANN 查询很少达到理想线性加速，因为 top-k merge、随机访存、cache miss 和线程调度都会随 P 增长。

Flat 的 T_{work} 最大，且每个候选距离计算相对规则，因此它最容易从线程数增加中获益。但 Flat 的 T_{mem} 也很大，因为每个 query 都要读取全部 base。PQ/FastScan 降低了 T_{work} ，使查表和内存访问变成主导；此时继续增加线程数，收益会比 Flat 更早饱和。IVF 同时降低 T_{work} 和 T_{mem} ，但会引入 coarse selection 和倒排链长度不均衡。HNSW 进一步降低访问候选数量，但其队列依赖导致单 query 内并行的 T_{sync} 和 T_{merge} 很高。

GPU 路径的模型略有不同。batch IVF 的设备端吞吐可以很高，但如果候选列表访问不连续、局部 top-k 选择复杂， T_{mem} 和 T_{merge} 仍会成为瓶颈。OpenMP target 和 SYCL 的实验都表明，GPU 优化不能只看并行线程数量，还要看候选裁剪是否足够强、selection 是否足够轻、batch 是否足够大。

这个模型解释了后文的主要结果：Flat 线程扩展明显但最终不快；FastScan 单候选很快但候选量仍偏大；IVF 在非图索引中最稳定；HNSW 由于候选访问量最低，最终在 CPU 上胜出；OpenMP target 能超过部分 CPU 非图路径，但无法超过 HNSW。

4 总体优化路线

本阶段的优化路线可以概括为“先压缩单次计算，再减少候选数量，最后选择合适并行粒度”。SIMD 阶段已经解决了单次距离计算的向量化问题；Pthread 阶段进一步面对的是调度、同步、内存访问和索引结构。图 1 给出了所有核心路径在 $\text{recall}@100 \geq 0.95$ 下的最低延迟，后续章节将逐一解释这些结果是如何得到的。

代码结构也按这一路线重构。距离计算核保持无状态，只接收 query 指针、base 指针和维度参数；索引层只负责生成候选集合，例如 Flat 的连续区间、IVF 的倒排链、PQ 的压缩 code 或 HNSW 的邻接点。CPU 多线程层采用 Pthread-style worker 模型：每个 worker 接收只读索引指针、query 步长、线程编号、线程私有 heap 和结果写入区，热循环中不持有锁、不共享 top-k 结构；OpenMP target 层则把候选矩阵、base 向量和距离输出展平成设备 kernel 可直接遍历的数组。这样可以避免把线程状态写进 SIMD 核，也使同一套 AVX2/NEON 距离函数能够复用于 Flat、IVF rerank、HNSW 节点距离和量化候选精排。

这一拆分还统一了后续实验的记录方式。每个算法族的 CSV 都保存算法名、线程数、核心参数、build time、平均 latency 和 $\text{recall}@100$ ；最终选点由脚本按 $\text{recall}@100 \geq 0.95$ 过滤后排序。报告中的图表不是手工录入结果，而是从这些结构化记录中重新绘制。也就是说，Pthread 线程策略、HNSW 查询内并行变体、OpenMP target 的 `nprobe/candidate cap` 调参和 SYCL 对比，最终都落到同一评价口径下。

从总体结果看，HNSW 图索引在 x86 和 ARM 上均取得最终最低延迟；IVF 是最稳定的非图索引；PQ/FastScan 通过量化和查表降低单候选成本，但在召回率目标下仍需较多候选；OpenMP target 与

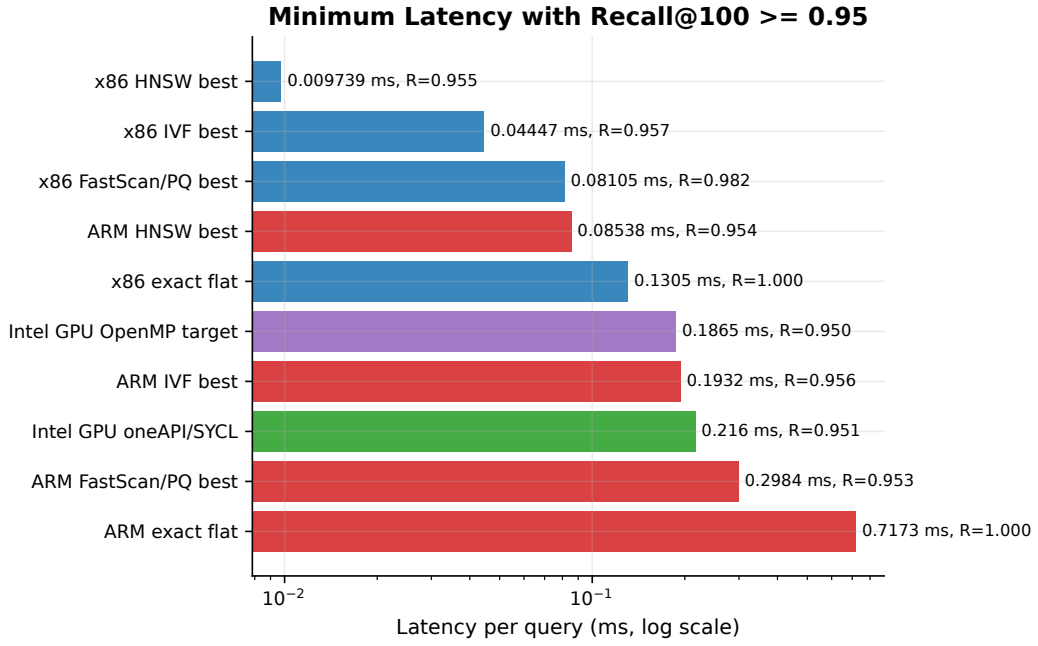


图 1: 核心算法在 $\text{recall@100} \geq 0.95$ 下的最低延迟。横轴为平均查询延迟，颜色区分 x86、ARM 与 Intel GPU 后端。

SYCL 的 GPU IVF 能跑出合理的近似搜索结果，但受限于随机访存和 top-k selection，未超过 CPU HNSW。

图 1 的横轴使用对数尺度，可以更清楚地观察不同算法族之间的数量级差异。x86 HNSW 位于最左侧，说明在本数据集上图导航减少候选访问的收益超过了继续优化单候选距离核；x86 IVF、FastScan 和 Flat 依次变慢，反映出“候选裁剪、查表压缩、全量扫描”三类策略的层次关系。ARM 上整体延迟更高，但排序基本一致，说明最终算法结构具有可迁移性；Intel GPU 的 OpenMP target 和 SYCL 位于 CPU IVF 附近，说明 batch scan 的吞吐已经发挥出来，但由于候选组织、距离矩阵回传和 host 端 selection 仍有开销，它们没有超过 CPU HNSW。

表 3 进一步给出各核心算法族的代表性可行点。这里没有把 recall 更高但延迟更大的配置作为最优，因为最终提交目标始终是 $\text{recall@100} \geq 0.95$ 下的最低 latency。这个选择规则会影响所有后续结论：Flat exact 和若干高 recall 的量化路径虽然结果更接近精确搜索，但它们消耗了过多候选扫描；HNSW 和 IVF 则通过控制访问范围，在满足目标线后立即停止继续扩大搜索。

表 3: 主要算法在 $\text{recall@100} \geq 0.95$ 下的代表性最优结果

平台	算法	关键参数	latency	recall@100	说明
x86 CPU	HNSW	StdAsync, ef=64, t=32	0.009739 ms	0.954540	全局最快 sweep 结果。
x86 CPU	IVF	n1512, nprobe=32, t=32	0.044471 ms	0.956950	非图索引最快。
x86 CPU	FastScan	block=64, p=750, t=32	0.081048 ms	0.981710	查表压缩路径。
x86 CPU	Flat exact	SSE, t=32	0.130451 ms	1.000000	精确搜索基线。
ARM CPU	HNSW	StdThread, ef=64, t=16	0.085378 ms	0.954280	ARM quick 最优。
ARM CPU	IVF	n1512, nprobe=32, t=32	0.193230 ms	0.956240	ARM 非图索引最快。
ARM CPU	FastScan	block=128, p=500, t=32	0.298396 ms	0.952520	NEON 下最优量化路径。
Intel GPU	OpenMP target IVF	nprobe=81, cap=3840	0.186502 ms	0.950170	严格设备卸载。
Intel GPU	oneAPI/SYCL IVF	nprobe=80	0.215992 ms	0.950600	GPU 编程工具对比。

5 Flat-SIMD 与 Pthread 并行

5.1 算法设计

Flat 搜索是所有 ANN 优化的起点。它对每个 query 枚举所有 base 向量，计算距离并维护 top-100。其优点是结果精确， $\text{recall}@100=1.0$ ；缺点是计算量和内存访问量最大。Pthread 阶段考虑两种并行方式：query-level 并行和 base-split 并行。

query-level 并行将不同 query 分配给不同线程。每个线程独立维护自己的 top-100 heap，线程间没有同步，适合 batch query 评测。base-split 并行把同一个 query 的 base 集合切成多个片段，每个线程维护局部 top- p ，最终归并为 top-100。这种方式能降低单 query 延迟，但 p 影响 recall：过小会丢失跨分片的真实近邻，过大会增加归并成本。

Flat 章节中的代码改造集中在任务切分和 top-k 合并。原 SIMD 阶段的距离函数只处理单线程扫描，Pthread 版本在其外层增加 **ThreadRange** 描述每个线程负责的 base 区间，并为每个线程预分配局部候选数组，避免在热循环中动态分配。query-level 路径直接把 query id 分块给线程；base-split 路径则要求每个线程输出局部 top- p ，主线程再执行一次小规模归并。为了减少 false sharing，线程私有候选缓冲按线程编号分段存放，不共享 heap 或计数器。

具体到 Pthread 调度，query-level 版本显式创建 T 个 worker，并使用循环式任务分配：第 t 个线程处理 $t, t+T, t+2T, \dots$ 号 query。每个 worker 只写自己负责的 result slot，因此不需要互斥量；主线程 join 后再统一统计 recall。base-split 版本先按线程数计算 base chunk，每个 worker 只扫描自己的连续区间，并把局部 heap 展开为向量保存到 `local\_results[t][qi]`。主线程归并时只处理 $T \times p$ 个候选，而不是重新访问原始 base，因此归并成本可由 p 显式控制。

这部分实现保留了精确搜索的可解释性，也暴露出 Pthread 优化的边界。距离计算已经由 SIMD 压缩后，Flat 的主要开销逐渐转向连续内存读取和 top-k 更新；base-split 虽然能并行单个 query，但局部候选越多，最终 merge 越重。因此 Flat 章节不仅是性能基线，也验证了后续 IVF/HNSW 中“先减少候选，再并行处理”的必要性。

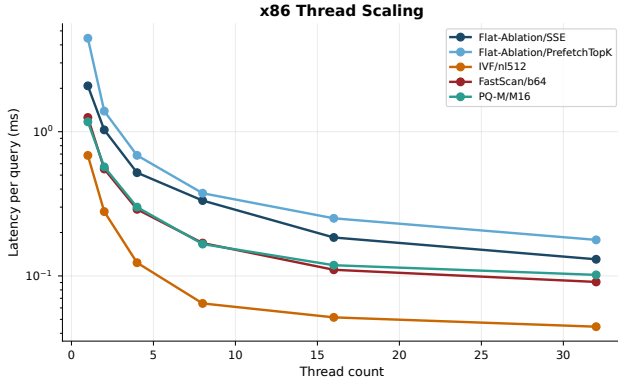
Algorithm 1 Flat base-split 并行搜索

- 1: 输入 query q , base 集合 X , 线程数 T , 局部候选数 p
 - 2: 将 X 均分为 T 个连续分片
 - 3: **for all** 线程 $t = 1, \dots, T$ **in parallel do**
 - 4: 在本地分片上用 SIMD 距离核扫描候选
 - 5: 维护局部 top- p
 - 6: **end for**
 - 7: 合并所有局部 top- p , 输出全局 top-100
-

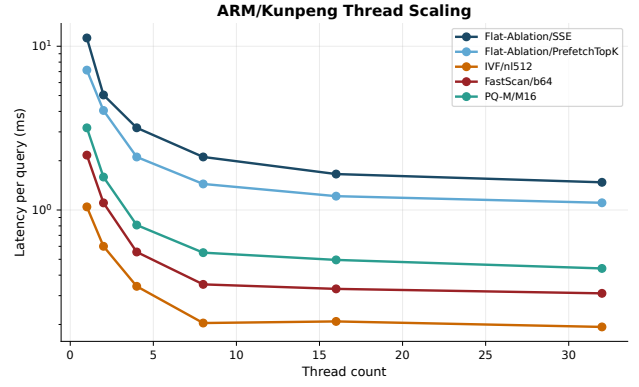
5.2 优化过程与实验结果

Flat 路径首先比较 Scalar、AutoVec、SSE、AVX2、NEON-Unroll4 和 PrefetchTopK 等距离核，再在 1、2、4、8、16、32 线程上消融。x86 上 32 线程 SSE 精确搜索达到 0.130451 ms/query；ARM 上最优精确 Flat 路径约 0.717344 ms/query。虽然多线程和 SIMD 都能显著降低 Flat 延迟，但它仍然慢于 IVF 和 HNSW，说明单纯提高吞吐无法替代候选裁剪。

从线程扩展曲线看，Flat 在 x86 上随线程数增加下降较明显，因为全量扫描的计算量足够大，worker 之间又几乎没有共享写入；但到 16 或 32 线程后，曲线已经不再接近线性下降，说明连续读



(a) x86 线程扩展



(b) ARM/Kunpeng 线程扩展

图 2: x86 与 ARM/Kunpeng 平台代表性算法的线程扩展曲线。Flat 随线程数下降最明显，量化、IVF 与 HNSW 更早进入访存、候选选择或调度瓶颈。

取 base 向量、维护 top-100 heap 和写回结果开始限制扩展。ARM 上的趋势更保守，主要原因是核心数、内存层次和 NEON SIMD 宽度与 x86 不同，同样的线程数未必对应同样的有效并行度。

Flat 实验给出两个后续优化方向。第一，全量扫描访问所有向量，查询开销由候选规模直接决定，因此仅靠 Pthread 把同样的 N 个候选分给更多线程，只能降低常数，不能改变复杂度。第二，线程数增加后收益逐渐下降，说明距离计算之外的内存访问、top-k 维护和调度开销开始占主导。因此后续算法重点转向减少候选量，而不是继续堆叠线程。这个判断直接引出了 SQ8/PQ/FastScan 的单候选压缩，以及 IVF/HNSW 的候选裁剪实验。

6 SQ8、PQ、SDC 与 FastScan

6.1 SQ8: 标量量化与 rerank

SQ8 将每个浮点维度量化为 8 bit 整数，查询时先用量化距离快速筛出候选，再用原始浮点向量 rerank。设原始向量维度为 D ，量化后每个维度只需整数运算和查表，单候选扫描成本明显降低。其 trade-off 在于候选数 p ：如果 p 太小，量化误差会使真实近邻被提前丢弃；如果 p 太大，rerank 成本会上升。

实现上，SQ8 路径把量化参数、压缩后的 base code 和原始 base 向量分开保存。第一阶段的近似扫描只读取 8 bit code 和 scale/zero-point，线程私有地维护近似 top- p ；第二阶段 rerank 再回到原始浮点向量，用 SIMD 距离核计算精确距离。这样可以把“近似筛选”和“精确重排”拆成两个可消融阶段，CSV 中分别记录候选数、线程数和最终 latency。Pthread 并行粒度仍以 query-level 为主，因为 SQ8 的 rerank 候选较少，强行在单 query 内继续切分会让 top-k 合并成本占比变高。

实验中 SQ8 在 x86 上最优可行点为 0.296953 ms/query, recall@100=0.989540；ARM 上最优点为 2.308882 ms/query, recall@100=1.0。这说明 SQ8 能保持高 recall，但在本数据集和实现中，候选筛选后的精排仍然偏重，不如 IVF 或 HNSW。

6.2 PQ-ADC 与 PQ-SDC

Product Quantization 将 D 维向量拆分为 M 个子空间，每个子空间用一个 codebook 表示。对 base 向量 x ，PQ 编码为

$$c(x) = (c_1(x), c_2(x), \dots, c_M(x)).$$

ADC (Asymmetric Distance Computation) 在查询时保留 query 的浮点值, 构建 query 到每个子空间 codeword 的 LUT, 然后对候选编码查表累加:

$$d_{\text{ADC}}(q, x) \approx \sum_{m=1}^M \text{LUT}_m[c_m(x)].$$

SDC (Symmetric Distance Computation) 同时量化 query 和 base, 进一步减少运行时浮点计算, 但量化误差更大。

Algorithm 2 PQ-ADC 查询

```

1: 输入 query  $q$ , PQ codebook, base codes, 候选数  $p$ 
2: for  $m = 1$  to  $M$  do
3:   计算子查询  $q_m$  到第  $m$  个 codebook 的 LUT
4: end for
5: for all base code  $c(x)$  in parallel do
6:    $s \leftarrow \sum_{m=1}^M \text{LUT}_m[c_m(x)]$ 
7:   用  $s$  更新近似 top- $p$ 
8: end for
9: 对 top- $p$  候选进行原始向量 rerank, 输出 top-100

```

PQ 的优势是把距离计算从 D 维浮点运算变为 M 次查表; 劣势是查表访问模式更随机, 线程数增加后可能很快被 cache 和内存带宽限制。因此本阶段重点消融 $M = 8, 12, 16$ 、候选数 p 和线程数, 而不是只报告单点结果。

PQ/SDC 的代码改造重点是 LUT 生命周期和线程私有化。ADC 中每个 query 都要构造 M 个子空间 LUT, 因此实现上把 LUT 放在线程本地缓冲, 避免多线程共享写入; base codes 采用连续数组存储, 使一个线程扫描候选时能顺序读取 code。SDC 路径则将 query 也编码后使用预计算表, 减少在线浮点计算, 但需要额外记录 query code 和 base code 的对应关系。Pthread 层只并行候选扫描和局部 top- p , 不并行单个 LUT 的构造, 因为 LUT 构建占比小, 拆得过细会增加调度开销。

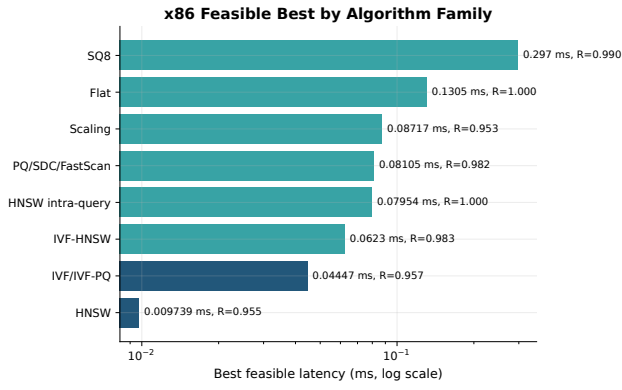
6.3 FastScan 与查表布局

FastScan 继续优化 PQ/SDC 的查表累加。核心思想是将 code 和 LUT 改成更适合 SIMD lane 的布局, 使多个候选的子空间查表可以在寄存器中并行累加。实验中 x86 的 FastScan 最优可行点为 block=64, p=750, threads=32, 延迟 0.081048 ms/query, recall@100=0.981710; ARM 上最优为 block=128, p=500, threads=32, 延迟 0.298396 ms/query, recall@100=0.952520。

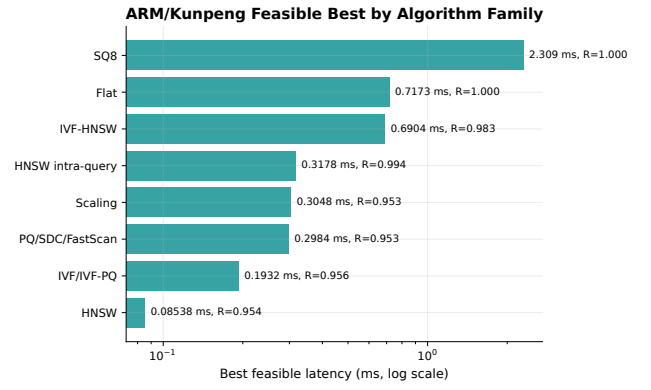
FastScan 的实现不再按“一个候选一段 code”的朴素布局扫描, 而是按 block 重排多个候选的子空间 code。这样一个 SIMD register 可以同时处理多个候选的同一子空间查表结果。Pthread 层按 block 区间切分任务, 每个线程处理完整 block, 避免两个线程写同一个 block 的局部分数。该实现使查表路径更接近顺序访问, 但也要求候选数和 block size 配合: block 太小无法充分利用 SIMD, block 太大则会增加缓存压力和局部 top- p 的维护成本。

从图 3 的两个面板可以看到, FastScan 在两个平台上都明显快于精确 Flat, 但没有超过 IVF 和 HNSW。原因是 FastScan 降低了单候选成本, 却没有像 IVF/HNSW 那样大幅减少候选数量。它适合作为 SIMD 与 Pthread 结合的高质量消融, 但不是最终最优路径。

两张算法族对比图还可以解释为什么本报告没有只保留 HNSW 结果。Flat 给出精确搜索上界和线程扩展基线; SQ8/PQ/SDC/FastScan 展示了压缩距离计算的收益与误差代价; IVF/IVF-PQ 展示了通



(a) x86 算法族最优点



(b) ARM/Kunpeng 算法族最优点

图 3: x86 与 ARM/Kunpeng 平台各算法族在 $\text{recall}@100 \geq 0.95$ 下的最优可行点。两张图使用同一评价口径，便于观察算法结构是否跨平台保持一致。

过 coarse quantizer 减少候选集合的效果；HNSW 查询内并行和 IVF-HNSW 则对应任务指导书中对索引并行的探索。最终 HNSW 胜出并不意味着其他实验可省略，因为只有通过这些消融才能说明性能提升来自图结构候选裁剪，而不是来自某个单独 SIMD kernel、线程数或计时口径。

6.4 量化路径的优化复盘

量化路径的优化过程体现了一个典型现象：当距离计算被压缩后，瓶颈会从 arithmetic 转向 memory 和 selection。SQ8 的每维量化非常直接，但为了修正量化误差，需要保留较多候选进行 rerank；PQ-ADC 的 LUT 构建开销较小，主要瓶颈是对 base codes 的随机查表；PQ-SDC 进一步减少查询时计算，却把误差引入 query 和 base 两侧；FastScan 通过改变数据布局缓解 SIMD 查表成本，但仍然无法减少需要扫描的候选总量。

代码层面，量化类方法的共同约束是线程不能共享临时 LUT、候选堆和 rerank 缓冲。否则即使距离核本身很快，也会在缓存一致性和锁竞争上损失性能。本文将这些状态全部放入 worker-local buffer：SQ8 的近似 top- p 和 rerank 输入数组由线程私有维护；PQ-ADC 的 LUT 按线程分配，query 切换时原地覆盖；FastScan 的 block 分配保证一个 block 只被一个线程处理。这样做牺牲了一定内存占用，但换来了清晰的并发边界和稳定的重复实验结果。

表 4: 量化与查表类方法的代表性结果

平台	方法	参数	latency	recall@100	主要解释
x86	SQ8	p=50, t=32	0.296953 ms	0.989540	recall 高, 但精排比例仍偏大。
x86	FastScan	block=64, p=750, t=32	0.081048 ms	0.981710	layout 优化后查表吞吐较好。
x86	FastScan scaling	block=64, p=500, t=32	0.087171 ms	0.952830	更接近目标 recall, 延迟略高于最优 IVF。
ARM	SQ8	p=200, t=32	2.308882 ms	1.000000	高 recall 依赖更多候选, 延迟不优。
ARM	FastScan	block=128, p=500, t=32	0.298396 ms	0.952520	ARM 上最优量化点, 但仍慢于 IVF。

从优化复盘看, PQ/FastScan 的价值不在于最终战胜图索引, 而在于展示 SIMD 与 Pthread 的组合边界。它们将单候选计算压到很低之后, 继续优化的方向应转向候选生成, 而不是继续微调 LUT 累加。因此后续进入 IVF 和 HNSW, 是算法结构层面的必要升级。

7 IVF 与 IVF-PQ

7.1 倒排索引结构

IVF (Inverted File) 先用聚类把 base 集合划分为 `nlist` 个簇, 并保存每个簇的质心和倒排链。查询时先计算 query 到所有质心的距离, 选择最近的 `nprobe` 个簇, 只扫描这些簇中的向量。其核心复杂度由全量 N 降到约 $N \cdot nprobe / nlist$, 代价是可能漏掉未被 probe 到的真实近邻。

IVF 的代码改造围绕倒排链数据结构展开。CPU 版本将每个 list 存为连续候选 id 数组, 并额外保存 list offset, 查询时先得到 `nprobe` 个 list id, 再把这些 list 的候选区间交给线程扫描。每个线程维护自己的局部 top-100, 完成后归并。这样的设计比在倒排链上加锁更直接, 因为查询阶段 list 只读, 写入只发生在线程私有候选堆中。IVF-PQ 复用同一套 list 结构, 只是候选距离由原始 SIMD 距离替换为 PQ 查表距离。

为了让 CPU 与 GPU 路径共享实验口径, IVF 的结果记录同时保存 `nlist`、`nprobe`、线程数和候选上界。x86/ARM 的 Pthread IVF 扫描完整倒排链; Intel GPU 的 OpenMP target IVF 在同一索引思想上增加 candidate cap, 用来控制设备端每个 query 的最大工作量。这使得 IVF 章节既承担非图索引实验, 也为后续 OpenMP target 加速器路径提供数据结构基础。

Pthread IVF 的 query-level worker 与 Flat 一致采用循环式 query 分配, 每个线程调用同一个 `ivf_search` 逻辑, 内部先计算 query 到质心的距离, 再扫描 `nprobe` 个倒排链。为了探索单 query 内并行, 代码还实现了 cluster-parallel fine scan: 先选出 `probe_ids`, 再让多个线程按 `pi = tid, tid+T, ...` 分担倒排链扫描, 每个线程维护本地 heap。最终结果仍要归并局部 heap, 因此它主要用于消融单

query 并行是否值得，而最终最优仍来自 query-level 并行。

Algorithm 3 IVF 查询

- 1: 输入 query q , 质心集合 C , 倒排链 $\mathcal{L}$, 参数 $nprobe$
 - 2: 计算 q 到所有质心的距离
 - 3: 选择最近的 $nprobe$ 个质心编号 P
 - 4: **for all** $j \in P$ **do**
 - 5: 扫描倒排链 $\mathcal{L}_j$ 中的候选
 - 6: 使用 SIMD 距离核或 PQ 近似距离更新 top-100
 - 7: **end for**
 - 8: 输出 top-100
-

7.2 $nprobe$ 的 trade-off

$nprobe$ 是 IVF 最关键的旋钮。增大 $nprobe$ 会提高 recall，但扫描候选数也近似线性增加。本文在 CPU 上消融 $nlist=64, 128, 256, 512$ 与 $nprobe=1, 2, 4, 8, 16, 32$ ，在 GPU 上围绕 $nlist=2048$ 细化 $nprobe=79..84$ 和 candidate cap。

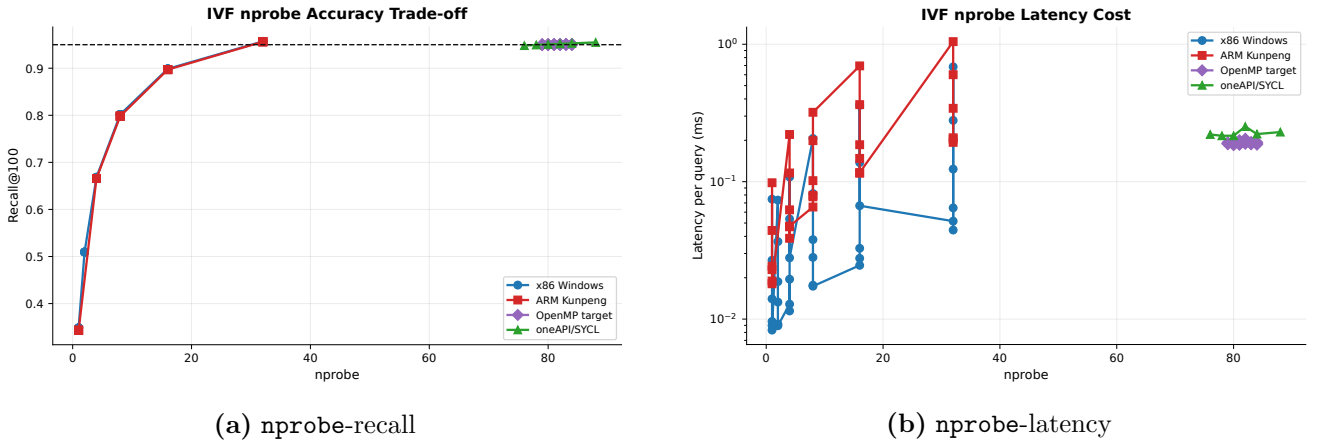


图 4: IVF 类方法的 $nprobe$ 敏感性。左图展示 recall 随 probe 数增加而提升，右图展示延迟随候选扫描量同步上升。

x86 CPU 上，最优 IVF 为 $nlist=512$, $nprobe=32$, $threads=32$, 延迟 0.044471 ms/query, $recall@100=0.956950$ 。ARM/Kunpeng 上最优 IVF 同样为 $nlist=512$, $nprobe=32$, $threads=32$, 延迟 0.193230 ms/query, $recall@100=0.956240$ 。这说明 IVF 的最佳 $nprobe$ 在两个平台间具有一定可迁移性，但绝对延迟受 SIMD 宽度、内存层次和线程运行时影响明显。

IVF 的实现重点在于让 $nprobe$ 的算法含义和线程执行边界一致。coarse 阶段只负责选择倒排链，不直接参与最终 top-100；fine scan 阶段才把候选 id 映射到原始向量或 PQ code，并在线程私有 heap 中维护局部结果。这样记录 CSV 时可以分别解释 recall 上升来自 probe 链增多，latency 上升来自 fine scan 候选增加。对于 GPU IVF，candidate cap 又进一步把这个候选量限制成固定矩阵形状，使 OpenMP target kernel 可以规则并行；代价是 cap 过小会截断候选、降低 recall，cap 过大又会增加设备扫描和回传距离矩阵的成本。

7.3 IVF-PQ 与局部 PQ

IVF-PQ 将“候选裁剪”和“量化距离”结合起来。全局 IVF-PQ 先对全体 base 训练 PQ，再在倒排候选内查表；局部 IVF-PQ 则在每个簇内训练或使用更贴合局部分布的 PQ。理论上，局部 PQ 能降低量化误差；实践中，它会增加构建成本，并且每个簇的训练样本更少，稳定性不如全局 PQ。

实验结果表明，IVF-PQ 系列是理解索引与量化叠加的重要消融，但在 DEEP100K 上最终最快的非图索引仍是 IVF + 原始距离精排。这说明当 base 规模为 100K、维度为 96 时，减少扫描簇数比进一步压缩单候选距离更重要。

8 HNSW 图索引

8.1 HNSW 查询过程

HNSW (Hierarchical Navigable Small World graph) 用多层近邻图组织 base 向量。查询从最高层入口点开始贪心下降，到 Layer 0 后维护候选队列和结果队列进行搜索。参数 `ef` 控制候选队列大小：较大的 `ef` 提高 recall，但会访问更多节点并增加 latency。

Algorithm 4 HNSW 标准查询

```
1: 从顶层入口点 ep 开始
2: for level 从最高层降到 1 do
3:   在当前层贪心移动到离 query 更近的邻居
4: end for
5: 在 Layer 0 维护候选队列 C 和结果队列 R
6: while C 中仍有可能改进 R 的节点 do
7:   弹出当前最近候选，访问其邻居
8:   若邻居更优则加入 C 和 R，并保持  $|R| \leq ef$ 
9: end while
10: 返回 R 中 top-100
```

8.2 query-level 并行

HNSW 查询内部存在强依赖：下一步访问哪个节点取决于当前候选队列状态。因此最稳定的多线程方式是 query-level parallel，即不同线程处理不同 query，共享只读图索引。x86 上 `StdAsync`，`ef=64`，`threads=32` 的 sweep 延迟为 0.009739 ms/query，`recall@100=0.954540`；ARM 上 `StdThread`，`ef=64`，`threads=16` 的 quick 延迟为 0.085378 ms/query，`recall@100=0.954280`。

实现上，HNSW 查询线程之间只共享图结构和向量数据，候选队列、结果队列、visited 标记都在线程私有内存中维护。x86 版本比较了 `std::async`、`std::thread` 和固定线程分块；ARM/Kunpeng 版本保留同样的逻辑，但默认 quick sweep 用较少 query 选择线程数，再用 final 路径验证。这个改造避免了多个线程竞争同一个候选队列，也让 HNSW 的 Pthread 优化集中在 query 分发、线程复用和私有缓冲复用上。

HNSW 的 visited 标记如果每次 query 都重新清空完整数组，会引入额外内存写入。因此实现中使用线程私有的访问版本号或局部访问列表，只清理实际访问过的节点。结果队列使用固定容量结构保存 `ef` 个候选，避免在搜索热路径中频繁申请内存。上述工程改造对延迟影响很大，因为最终 HNSW 单 query 访问节点数已经很少，任何线程调度和内存初始化开销都会被放大。

在标准 HNSW query-level 版本中，`std::thread` 路径显式创建 T 个 worker，并按 query id 间隔分配任务；`std::async` 路径把同样的分配方式交给 C++ 运行时调度，并用 future 等待所有任务完成。每个 query 调用 `searchKnn` 后，将 hnsplib 返回的 heap 转换为统一的 ANN top-k 结构，再计算 `recall@100`。这保证 HNSW、IVF 和 Flat 的结果评测函数一致，避免因为不同库返回顺序不同而影响 recall 统计。

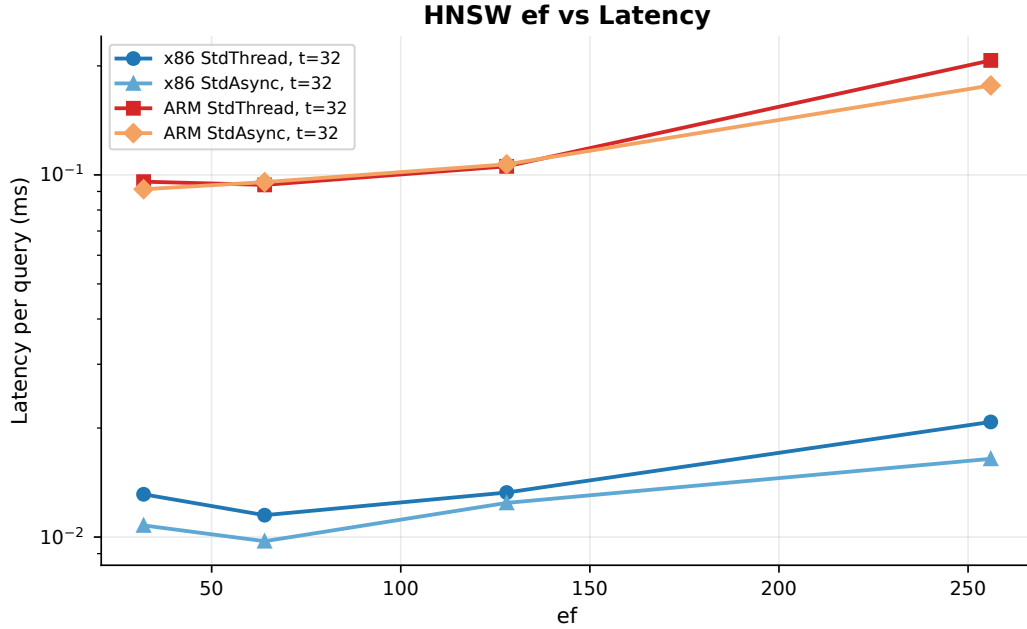


图 5: HNSW 中 `ef` 对延迟的影响。满足 recall 后继续增大 `ef` 会迅速增加搜索成本。

从图 5 可以看出，`ef=64` 是两个平台上兼顾 recall 和 latency 的稳定点。更大的 `ef` 提高 recall，但已经偏离 `recall@100 ≥ 0.95` 下最小 latency 的目标。

8.3 查询内并行探索

任务指导中提到的图索引查询内并行包括边划分、Layer 0 点划分、多入口点并行，以及 IVF+HNSW 的嵌套结构。本文分别实现并测试了这些策略。其共同思路是把一个 query 的 Layer 0 搜索拆给多个线程，再合并局部结果。

边划分版本把当前扩展节点的邻接边分给多个线程计算距离，然后合并可进入候选队列的邻居；点划分版本把 Layer 0 候选点集合划成多个子集，各线程在子集内维护局部队列；多入口点版本从多个入口同时启动搜索，最后归并局部 top-100；IVF+HNSW 则先用 IVF 粗分簇，再在每个子簇内部运行一个较小的 HNSW。代码上这些变体都使用线程私有队列和 visited 标记，避免共享队列加锁成为唯一瓶颈。它们的差别主要体现在任务构造方式和最终候选归并方式。

Algorithm 5 Layer 0 查询内并行的一般形式

- 1: 输入 query q , HNSW Layer 0 图, 线程数 T
 - 2: 按入口点、边集合或点集合构造 T 个子任务
 - 3: **for all** 子任务 t **in parallel do**
 - 4: 在子图或子候选集合上执行局部搜索
 - 5: 输出局部 top-100 或局部候选队列
 - 6: **end for**
 - 7: 合并所有局部结果, 并计算最终 top-100
-

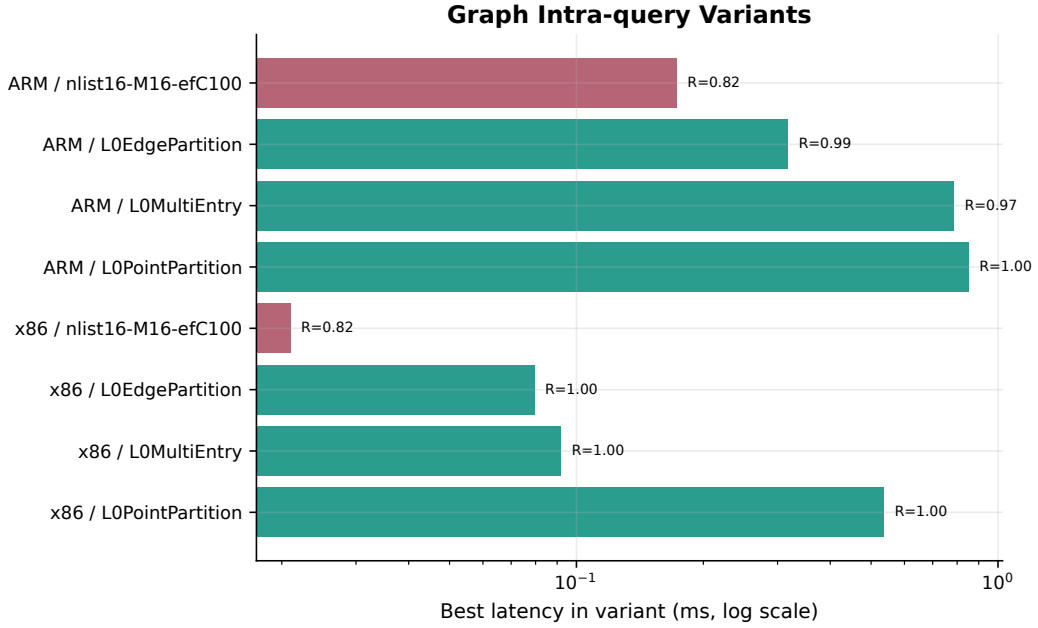


图 6: HNSW 查询内并行和 IVF-HNSW 变体的最快点。绿色满足 $\text{recall}@100 \geq 0.95$, 紫红色不满足目标线。

图 6 显示, 查询内并行并没有超过标准 query-level HNSW。部分变体能获得高 recall, 但延迟明显更高; 少数低延迟点 recall 只有约 0.82, 不能作为最终候选。这一现象符合 HNSW 的算法结构: 图搜索的价值来自自适应剪枝, 简单切分点或边会削弱队列驱动的导航过程, 并引入局部队列合并开销。

表 5: HNSW 查询内并行变体的最快记录

平台	变体	参数	latency	recall@100	解释
x86	Layer0EdgePartition	ef=128, t=4	0.079540 ms	0.999820	recall 高, 但慢于标准 HNSW。
x86	Layer0MultiEntry	ef=128, entries=2	0.091911 ms	1.000000	多入口增加覆盖, 也增加重复搜索。
x86	Layer0PointPartition	partitions=8	0.536787 ms	1.000000	近似退化为分区精扫, 延迟高。
x86	IVF-HNSW	nprobe=2, t=32	0.020957 ms	0.817610	延迟低但 recall 不达标。
ARM	Layer0EdgePartition	ef=128, t=8	0.317768 ms	0.994480	满足 recall, 但同步与合并较重。
ARM	IVF-HNSW	nprobe=2, t=16	0.173162 ms	0.815660	子图过少导致召回不足。

这些结果不是无效实验, 而是说明图索引查询内并行的难点。若希望真正提升 HNSW 单 query 并

行性能，简单划分 Layer 0 还不够，需要重新设计 visited 标记、候选队列共享方式和终止条件；否则线程间并行会增加大量重复访问，反而削弱 HNSW 的导航优势。

9 OpenMP Target 与 oneAPI/SYCL

9.1 加速器实现目标

OpenMP 进阶要求的重点是卸载到加速器设备，因此本文不把普通 CPU OpenMP pragma 作为该部分的完成证据。GPU 路径采用 batch IVF：host 端完成质心训练、倒排链构建和 candidate 填充，device 端负责批量 query 的候选距离扫描，host 端回收距离矩阵后完成最终 top-k 选择。这样可以把 GPU 的规则并行能力集中用在最重的候选扫描上，同时避免把 HNSW 这类动态优先队列搜索直接搬到设备端。

代码实现中，质心矩阵、倒排链 offset、候选 id 数组和 base 向量被整理为连续内存，并在 target data 区域映射到设备。每个 batch 先在 host 侧根据 `nprobe` 和 `candidate cap` 填充候选 id；随后 target kernel 展开为 `q\_count * candidate\_cap` 个并行迭代，每个迭代计算一个 query-candidate 距离。无效候选槽写入极大距离，保证输出距离矩阵形状固定，便于 host 端统一执行 top-k selection 和 recall 统计。

9.2 OpenMP target IVF

OpenMP target 实验严格卸载到 Intel GPU 加速器设备，运行时设置 mandatory offload，并在 target 区域调用 `omp\_is\_initial\_device()` 检查当前执行设备不是 host。算法上采用 `nlist=2048` 的 IVF，围绕 `nprobe` 和 `candidate cap` 进行精细搜索。与 CPU Pthread 不同，OpenMP target 的主要优化不是增加 host 线程，而是控制设备端扫描候选数、提高 batch 内并行度，并把 scan、selection 和回传三段时间分开统计。

这部分代码刻意保持 kernel 简单：设备端不使用 STL 容器、不动态申请内存、不维护复杂优先队列，而是将二维任务空间展平为 `q\_count * candidate\_cap` 个迭代。每个迭代只做三件事：读取候选 id，判断 slot 是否有效，计算一个 query 与一个 base 向量的距离并写入距离矩阵。这样的写法更接近 GPU 的规则数据并行模型，也更容易在 Intel OpenMP target 编译链下稳定使用普通 `-O2`。最终 top-100 selection 留在 host 端完成，是为了先保证结果正确、便于和 CPU IVF 复用同一 recall 统计函数；报告中也把 selection 作为后续可优化瓶颈单独讨论。

Algorithm 6 OpenMP target batch IVF

- 1: Host 构建 IVF 质心、倒排链和紧凑候选数组
 - 2: Host 按 `nprobe` 和 `candidate cap` 填充 batch 候选矩阵
 - 3: 将 batch queries、candidate ids 和 base vectors 映射到 target device
 - 4: **for all** device 端的 (*query*, *candidate slot*) 并行迭代 **do**
 - 5: 读取候选 id；无效 slot 写入极大距离
 - 6: 对有效候选执行 SIMD-friendly 距离累加，写出距离矩阵
 - 7: **end for**
 - 8: 将距离矩阵回传 host
 - 9: Host 完成 final top-100 selection 与 recall 统计
-

最优 OpenMP target 配置为 `nprobe=81`, `candidate cap=3840`, 延迟 0.186502 ms/query,

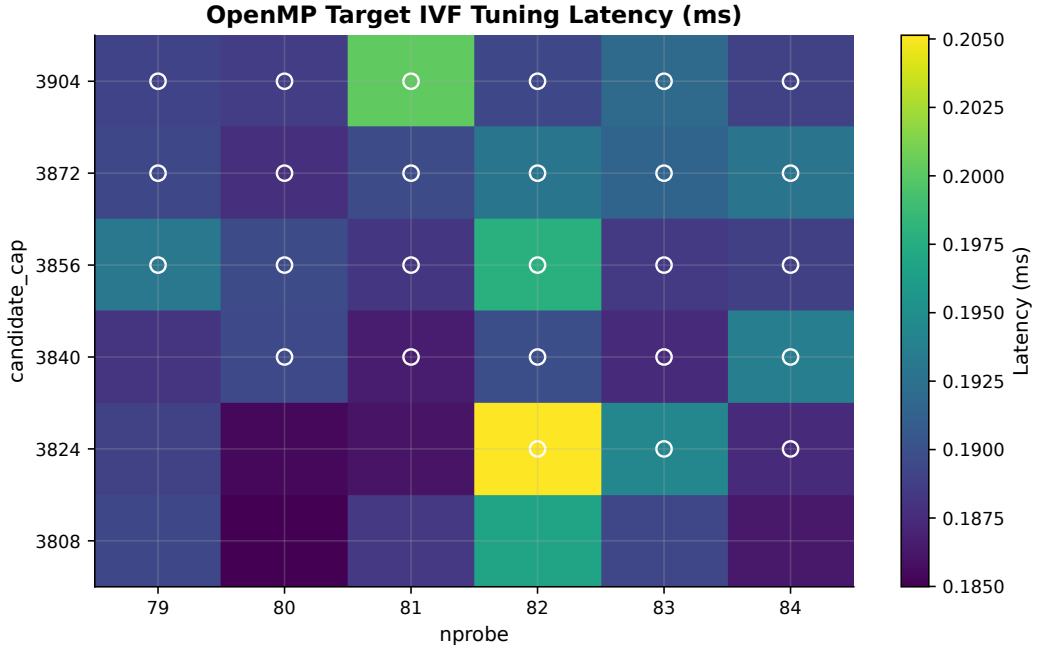


图 7: OpenMP target IVF 的 $nprobe$ 与 $candidate_cap$ 调参热力图。白色圆圈为满足 $recall@100 \geq 0.95$ 的可行点。

$recall@100=0.950170$ 。这个点非常接近目标线，说明优化方向已经从“让 GPU 跑起来”转为“减少不必要候选扫描”。

从实现角度看，OpenMP target 路径的关键修改有三点。第一，倒排链不使用 CPU 侧的指针结构，而是改为 `offsets + ids + vectors` 的扁平数组，便于设备端按 `offset` 定位候选区间。第二，kernel 内部避免动态分配和复杂 STL 容器，每个并行迭代只做一次连续向量点积并写回距离，保证设备端代码可被普通 `-O2` 编译。第三，`candidate cap` 既是算法参数，也是设备端工作量和回传距离矩阵的上界，可以防止少数超长倒排链拖慢整个 batch。

这一实现把 OpenMP target 的收益和代价都显式暴露出来。设备端扫描时间对应 `target teams distribute parallel for` 的实际工作，`candidate` 填充时间反映 host 侧根据倒排链构造固定形状候选矩阵的成本，`selection` 时间反映回到 host 后维护 top-100 的成本。图 8 中可以看到，GPU 路径并不是单一 kernel 时间，而是由 `candidate` 组织、设备扫描、距离回传和 top-k 选择共同决定。后续优化空间也很明确：若要进一步提升 OpenMP target，需要把 block-level top-k 或 partial selection 下沉到设备端，减少回传距离矩阵规模。

9.3 与 oneAPI/SYCL 的比较

oneAPI/SYCL 路径采用相近的 Intel GPU batch IVF 思路。正式 O2 结果中，SYCL 最优为 $nprobe=80$ ，延迟 0.215992 ms/query， $recall@100=0.950600$ 。它略慢于 OpenMP target 最优点，但两者的瓶颈一致：候选 scan 和 top-k selection 是主要耗时。

GPU 结果说明：OpenMP target 能完成“卸载到加速器设备”的进阶要求，但在本数据规模下不能超过 x86 CPU HNSW。原因不是 GPU 一定慢，而是本实验的最优 ANN 结构是图索引，图搜索的随机依赖并不适合直接映射到当前 Intel 集显；而 GPU-friendly 的 IVF 路径虽然并行度高，但候选裁剪能力不如 HNSW。

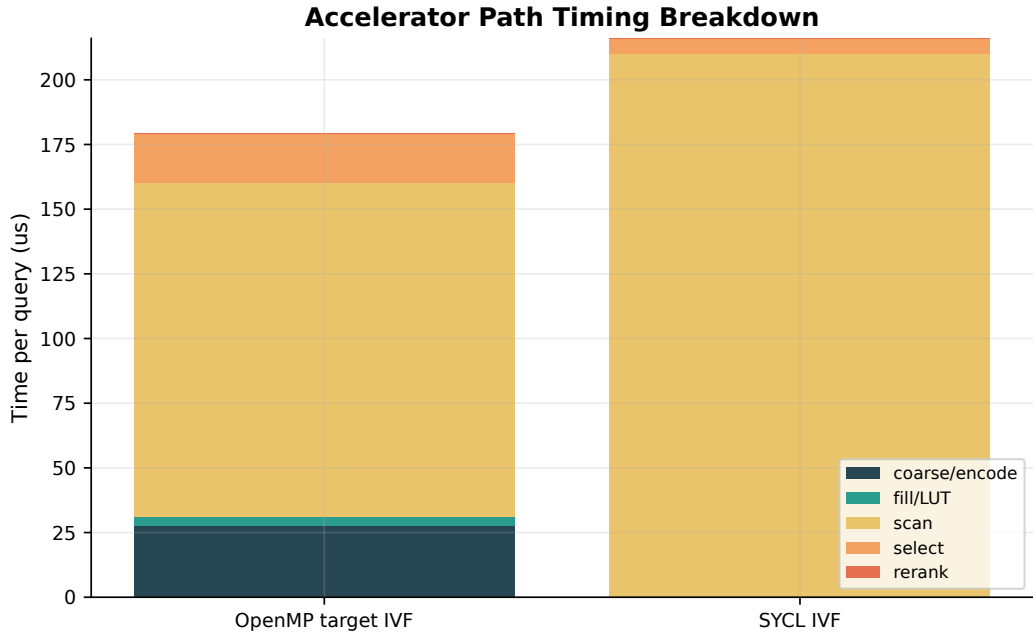


图 8: OpenMP target 与 oneAPI/SYCL 的阶段耗时分解。GPU 路径的主要开销集中在候选扫描和选择。

表 6: Intel GPU 上 IVF 加速器路径对比

后端	参数	latency	recall@100	build(s)	分析
OpenMP target	nprobe=81, cap=3840	0.186502 ms	0.950170	10.386833	最接近目标线，候选扫描和 selection 是主要开销。编程模型更灵活，但本实验中略慢于 OpenMP target。
oneAPI/SYCL	nprobe=80, cap=9929	0.215992 ms	0.950600	19.498300	
OpenCL C	nprobe=88	0.260130 ms	0.954270	279.108000	
CUDA Flat	exact scan	0.410661 ms	1.000000	0.000000	作为底层 GPU API 参照，构建和调参成本更高。精确扫描基线，未作为最终加速器路径。

这一对比也解释了为什么报告不把 OpenMP CPU pragma 作为主要结果。进阶实验的关键是“卸载到加速器设备”，因此 OpenMP 的正式结果必须来自 target device；CPU 多线程则由 Pthread 和 C++ 标准库工具完成。这样的划分使实验结论更清晰：CPU 上最优是 HNSW，GPU 上最适合展示的是 batch IVF。

10 x86 与 ARM/Kunpeng 跨平台分析

10.1 算法最优点是否迁移

跨平台对比中, IVF 的最优参数具有较强一致性: x86 和 ARM 都选择 `n1512`, `nprobe=32`。这说明在 DEEP100K 上, 倒排簇数量与 recall 目标之间的关系主要由数据分布决定, 而不是平台决定。不同平台影响的是绝对 latency: ARM IVF 为 0.193230 ms/query, 约为 x86 IVF 的 4.35 倍。

HNSW 的最优算法族也一致, 但最佳线程工具和线程数发生变化。x86 上 `std::async` 与 32 线程更快; ARM 上 `std::thread` 与 16 线程更稳。这说明当单 query 代价已经很低时, 线程运行时开销、核心数和调度策略会影响最终选择。

10.2 构建时间与查询延迟

索引构建时间不计入单次查询 latency, 但对实际系统部署很重要。Flat 无索引构建, 但查询慢; IVF 需要 KMeans 聚类, 构建时间中等; HNSW 构图更重, 但查询最快。图 9 展示了构建代价和查询延迟之间的关系。

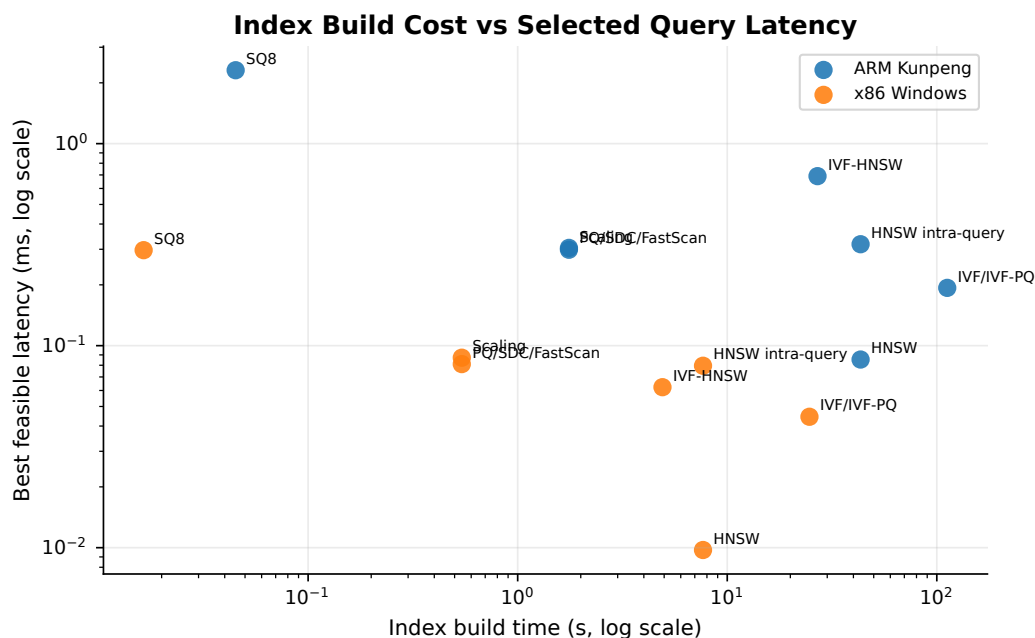


图 9: 索引构建时间与最优查询延迟的关系。HNSW 用更高构建成本换取最低查询延迟。

表 7: 最终全量提交路径

平台	算法	线程	参数	latency(μ s)	recall@100	build(s)
x86 Windows	HNSW-StdAsync	32	ef=64	9.316000	0.952683	6.238946
ARM/Kunpeng	HNSW-StdThread	16	ef=64	70.878600	0.952643	43.452011

最终路径与 sweep 结论一致: 两个平台都选择 HNSW, 但工具和线程数按平台调整。特别是 ARM 结果已经通过全量 `test.sh` 验证, 不只是 quick sweep 的估计。

10.3 最终参数选择依据

最终选择 `ef=64` 而不是更大的 `ef=128` 或 `ef=256`，是因为目标函数已经固定为 $\text{recall}@100 \geq 0.95$ 下的最低 latency。以 x86 HNSW 为例，`ef=64` 已达到 $\text{recall}@100=0.954540$ ；继续增大 `ef` 可以提高 recall，但需要扩展更多图节点，平均延迟随之上升。ARM 上也出现相同趋势，因此 `ef=64` 是两个平台都稳定跨过目标线的最低成本点。

线程数也不是越高越好。x86 本地 CPU 的逻辑线程更多，`std::async` 在 32 线程下调度表现最好；ARM/Kunpeng 实机为 8 核环境，16 线程的 `std::thread` 最稳。若强行沿用 x86 的 32 线程 `std::async`，ARM 上会引入更明显的调度和上下文切换开销。最终路径按平台分开选择，是跨平台实验的直接结论，而不是手工偏置。

表 8: 最终路径与主要备选路径比较

平台	方法	latency	recall@100	取舍分析
x86	HNSW final	0.009316 ms	0.952683	全量最终路径，最低延迟。
x86	IVF best	0.044471 ms	0.956950	非图索引最快，但约为 HNSW 的 4.8 倍。
x86	FastScan best	0.081048 ms	0.981710	recall 更高，但不满足最低延迟目标。
ARM	HNSW final	0.070879 ms	0.952643	全量最终路径，优于 ARM quick 中其他算法族。
ARM	IVF best	0.193230 ms	0.956240	稳定但候选扫描多于 HNSW。
Intel GPU	OpenMP target IVF	0.186502 ms	0.950170	严格卸载到设备，快于 ARM IVF，但慢于 CPU HNSW。

表 8 说明最终提交路径并不是只看某一个算法的局部结果，而是在所有满足 recall 约束的候选中比较得出。HNSW 的优势来自候选访问量的数量级减少；IVF 的优势是结构简单、参数可解释；FastScan 的优势是单候选计算快；OpenMP target 的优势是展示了加速器卸载能力。最终选择 HNSW，是因为它在目标指标下确实给出了最低 latency。

11 综合讨论：算法结构与硬件匹配

11.1 从计算密集到访存密集

SIMD 阶段主要解决的是“单次距离计算如何更快”，而 Pthread 阶段暴露出一个更重要的问题：当单候选距离已经足够快时，系统瓶颈会逐渐从算术计算转向访存、候选选择和线程协作。Flat 搜索最接近 compute-bound，因为每个候选都要计算完整距离，线程扩展和 SIMD 都能直接降低主要计算时间；但 Flat 的候选数固定为 N ，它无法从算法结构上减少访问量。

SQ8、PQ、SDC 和 FastScan 将单候选计算压缩为量化、查表或整数累加。它们在单候选层面比 Flat 更轻，但也更容易进入 memory-bound 状态：查表地址由压缩 code 决定，访问模式不如连续浮点数组规则；多个线程同时扫描 code 时，cache miss 和内存带宽会限制扩展。FastScan 通过更适合 SIMD lane 的布局缓解了这一点，但无法改变“仍要扫描大量候选”的事实。因此，量化路径的优化上限由候

选规模和访存模式共同决定。

IVF 和 HNSW 的核心优势是减少候选数量。IVF 使用质心先过滤倒排链，使候选量约为 $N \cdot nprobe/nlist$ ；HNSW 则使用图导航，仅访问与 query 更相关的一小部分节点。二者都从算法结构上减少了后续距离计算和 top-k 维护，因此即使单候选计算未必比 FastScan 更轻，端到端 latency 仍然更低。最终结果显示，HNSW 的候选裁剪能力最强，因此在 CPU 平台上成为最优路径。

11.2 线程粒度的选择原则

线程粒度决定了并行化是降低主要工作量，还是引入额外同步。对 Flat 和 IVF，query-level 并行最稳定，因为每个 query 的 top-k 结构可以独立维护，线程间只共享只读数据。base-split 和倒排链内部分片虽然能降低单 query 延迟，但会引入局部 top-k merge；当候选数不够大或线程数过高时，merge 开销会抵消收益。

对 HNSW，query-level 并行几乎是最自然的选择。标准 HNSW 查询依赖候选队列的动态状态，当前扩展出的节点会决定下一步访问哪些邻居。Layer 0 多入口、边划分和点划分都试图把这一过程拆开，但拆开之后每个线程都缺少全局队列视角，容易产生重复搜索或过早访问不重要节点。实验中这些变体能作为图索引并行探索，但没有超过标准 query-level HNSW，说明图搜索算法的同步结构比 Flat/IVF 更难并行。

表 9: 不同并行粒度的收益与代价

并行粒度	适用算法	主要收益	主要代价
query-level	Flat、PQ、IVF、HNSW	线程间状态独立，几乎无 top-k 归并	需要 batch queries；单 query latency 不一定下降。
base-split	Flat exact	降低单 query 全量扫描时间	局部 top-p 归并影响 recall 和 latency。
倒排链分片	IVF/IVF-PQ	对长 list 可提升负载均衡	list 长度不均，候选归并复杂。
LUT/子空间并行	PQ-ADC	LUT 构建阶段规则、易并行	LUT 构建本身占比小，收益有限。
Layer 0 查询内并行	HNSW	探索单 query 内并行可能性	破坏全局队列剪枝，重复访问和同步开销高。
GPU batch 并行	OpenMP target/SYCL IVF	大批量候选扫描吞吐高	top-k selection、随机访存和 host-device 协作限制收益。

11.3 近似误差与参数敏感性

ANN 算法的误差来源并不相同。量化类方法的误差主要来自向量表示压缩，SQ8 将每一维映射到整数区间，PQ 将原向量拆成子空间并使用码本近似，FastScan 则进一步把查表布局改造成更适合 SIMD 的形式。它们的 recall 变化通常比较平滑：候选数 p 、rerank 规模或码本粒度增大时，返回集合更接近精确 top-k，但查询阶段要处理更多候选和查表访问。

IVF 的误差主要来自 coarse quantizer 的簇选择。若 query 附近的真实近邻分散在多个倒排链中，较小的 $nprobe$ 会漏掉部分近邻；随着 $nprobe$ 增大，扫描链数和候选数同步上升，recall 与 latency 呈现非常直接的单调关系。HNSW 的误差则来自图导航路径， ef 控制候选队列宽度，较大的 ef 能让搜索保留更多备选路径，但也增加邻接点访问、优先队列维护和 cache 压力。

这些参数敏感性决定了消融实验的组织方式。量化类方法需要观察候选数与精排成本，IVF 需要观察 `nprobe` 对倒排链访问量的影响，HNSW 需要观察 `ef` 对访问节点数和线程调度的影响，GPU batch IVF 还要额外关注每批 query 的负载均衡、距离矩阵回传和 host 端 top-k 选择成本。只有把误差来源和硬件开销对应起来，才能解释为什么某些参数在 x86 上有效，却不一定能迁移到 ARM 或 GPU。

11.4 跨平台差异的解释

x86 与 ARM 的最优算法族相同，但线程选择不同。IVF 在两个平台上都选择 `n1512`，`nprobe=32`，说明该参数主要由数据分布和 recall 目标决定；ARM 的绝对延迟更高，则反映了单核性能、SIMD 宽度和内存层次差异。HNSW 在两个平台上都最优，但 x86 使用 `std::async` 与 32 线程，ARM 使用 `std::thread` 与 16 线程，说明单 query 成本较低时，线程运行时开销会成为决定性因素。

Intel 集显适合规则 batch 扫描，但 HNSW 的图搜索依赖和随机分支不适合直接卸载。本文因此选择 GPU-friendly 的 IVF。OpenMP target IVF 达到 0.186502 ms/query，快于 ARM IVF，但仍慢于 x86 HNSW，说明设备并行只有在算法结构与硬件执行模型匹配时才会转化为端到端收益。

11.5 对最终方案的工程判断

最终提交路径选择 HNSW query-level 并行，因为它在两个 CPU 平台上都给出最低 latency，并且全量查询 recall 稳定超过 0.95。若数据频繁更新，IVF 的构建和查询平衡可能更合适；若关注 GPU 批量吞吐，OpenMP target/SYCL IVF 仍有继续优化空间；若强调 SIMD + Pthread 的教学可解释性，FastScan 和 PQ 提供了清晰的单候选优化过程。

因此，最终结论不是某一种线程接口“最好”，而是算法结构决定上限，硬件并行决定能否接近上限。候选数量已经减少后，再选择低同步开销的并行粒度，才能得到稳定的跨平台性能。

12 优化过程总结

本阶段优化过程可以按四个阶段理解。第一阶段是并行精确搜索，Flat-SIMD + Pthread 验证了多线程能降低精确搜索延迟，但仍要访问全部 base，因此主要作为 recall 上界和 speedup 基线。第二阶段是压缩单候选计算，SQ8、PQ、SDC 和 FastScan 将浮点距离转为量化、查表和整数累加；FastScan 在 x86 上达到 0.081048 ms/query，但由于候选规模仍大，最终慢于 IVF 和 HNSW。

第三阶段是减少候选数量。IVF 通过 `nprobe` 控制倒排链扫描范围，是最快的非图索引；HNSW 进一步利用图导航避免大量无关候选，成为最终最优路径。第四阶段是匹配硬件并行模型：CPU 上 query-level parallel 最稳定，HNSW 查询内并行因破坏图搜索依赖而负优化；GPU 上必须使用 batch IVF 这类规则并行结构来发挥吞吐优势。

13 结论

本文完成了基于 SIMD 的 ANN 多线程与加速器并行实验。所有结果统一使用 `recall@100`，并以 `recall@100 ≥ 0.95` 下最小 latency 为唯一最优准则。x86 和 ARM 的最终最优均为 HNSW query-level 并行，说明图索引候选裁剪比单纯增加线程数更关键。IVF 是稳定的非图索引；PQ/FastScan 能降低单候选成本，但受候选规模和访存限制。OpenMP target 已卸载到 Intel GPU，batch IVF 满足 recall 约束，但本实验规模下仍慢于 CPU HNSW。

A 完整图表与复现说明

报告中的所有图均由真实实验 CSV 重新绘制，每张图只表达一个分析问题，不使用拼接子图。图表生成命令如下：

```
cd D:\Parallel-programming-NKU\ann
python plot_pthread_report.py

cd D:\Parallel-programming-NKU\ann\report
xelatex -interaction=nonstopmode pthread_report.tex
xelatex -interaction=nonstopmode pthread_report.tex
```

表 10: 报告图表含义

图编号	分析内容
图 1	各核心算法在 recall 约束下的最低延迟。
图 2、2	代表算法的线程扩展趋势。
图 3、3	不同算法族在 x86 和 ARM 上的最优可行点。
图 4、4	IVF 的 <code>nprobe</code> 对 recall 和 latency 的影响。
图 5、6	HNSW 的 <code>ef</code> 消融与查询内并行探索。
图 7、8	OpenMP target 与 SYCL 加速器路径分析。
图 9	构建成本与查询延迟 trade-off。

B 生成式 AI 辅助学习附录

本附录整理实验过程中借助生成式 AI 形成的知识性问答。问答重点放在 ANN 算法、多线程并行、SIMD 与加速器卸载的概念理解上，而不是记录具体代码修改指令。每组内容均采用“用户问题 / AI 回答”的形式，便于作为课程报告附录阅读。

问答 1: ANNS 与精确 kNN 的本质差别是什么

用户问题： 近似最近邻搜索和精确 kNN 的核心差别是什么？

AI 回答： 精确 kNN 要在全体 base 向量中找到距离最小的 k 个点，通常意味着对所有候选进行距离计算和排序维护。ANNS 则通过索引结构、量化表示或图导航减少候选访问量，在给定 recall 约束下换取更低延迟。它的关键不只是让单次距离计算更快，而是尽量避免访问与 query 无关的大量向量。

问答 2: 为什么 SIMD 优化后仍然需要多线程

用户问题： 如果 SIMD 已经加速了距离计算，Pthread 多线程还有什么意义？

AI 回答： SIMD 提升的是单个线程内部的数据级并行，通常把一组维度的乘加或查表压到少数指令中完成。Pthread 提供的是任务级或数据块级并行，可以同时处理多个 query、多个倒排链或多个候选分片。二者解决的问题层次不同：SIMD 降低单候选计算时间，多线程扩大同时处理的候选或 query 数量。最终收益取决于算法瓶颈是计算、访存、同步还是索引遍历。

问答 3: Flat 搜索为什么适合作为基线

用户问题: Flat 搜索在 ANN 实验中有什么基线价值?

AI 回答: Flat 搜索直接扫描全部 base 向量, 逻辑简单、误差来源少, 适合衡量 SIMD 核、线程调度和 top-k 维护的基础开销。它可以提供延迟上界和实现正确性的参照。与索引类方法相比, Flat 不依赖构建质量或参数选择, 因此能清楚反映硬件计算吞吐和内存带宽对精确扫描的影响。

问答 4: SQ8 量化的误差来自哪里

用户问题: SQ8 为什么能加速距离计算, 它的误差来自哪里?

AI 回答: SQ8 将浮点向量的每一维压缩为 8 位整数, 减少存储和访存量, 也使距离计算可以使用更轻量的整数运算。误差来自量化间隔: 连续浮点值被映射到离散整数桶后, 原始距离会产生近似偏差。若数据分布较平滑, SQ8 能以较小精度损失换取更高吞吐; 若近邻之间距离非常接近, 量化误差可能改变候选排序。

问答 5: PQ 与 SQ8 的主要区别是什么

用户问题: Product Quantization 和 SQ8 都是量化, 它们的思路有什么不同?

AI 回答: SQ8 通常逐维量化, 每个维度独立映射到整数区间。PQ 则把向量划分成多个子空间, 每个子空间用一个码本代表局部向量块, 最终用多个 codeword 拼接近似原向量。PQ 的压缩率更高, 适合大规模候选粗筛; 但距离计算依赖查表累加, 误差由子空间划分、码本训练质量和 code 长度共同决定。

问答 6: ADC、SDC 和 FastScan 的关系是什么

用户问题: PQ 中的 ADC、SDC 和 FastScan 分别解决什么问题?

AI 回答: ADC 使用原始 query 与压缩后的 base code 计算近似距离, 通常为每个 query 构建 lookup table, 再对候选 code 查表累加。SDC 使用压缩 query 与压缩 base 之间的预计算距离, 降低在线计算量但会引入额外近似。FastScan 关注查表访问的布局 and SIMD 友好性, 通过组织 code block 和低位表项减少随机访存, 让查表累加更适合向量化执行。

问答 7: IVF 的 coarse quantizer 起什么作用

用户问题: IVF 为什么要先做 coarse quantization?

AI 回答: IVF 将 base 向量按质心划分到多个倒排链中。查询时先找到离 query 最近的一批质心, 只扫描对应倒排链, 而不是扫描全部 base。coarse quantizer 的作用是把全局搜索转化为局部候选扫描; 它的质量决定真实近邻是否集中在被访问的倒排链中, 也决定候选量和负载均衡。

问答 8: HNSW 为什么适合低延迟查询

用户问题: HNSW 为什么常常能在 ANN 中取得很低查询延迟?

AI 回答: HNSW 使用分层小世界图组织向量。高层图提供长距离跳转, 能快速接近 query 所在区域; 底层图提供更密集的局部邻接, 用于细化搜索。查询过程维护候选队列和已访问集合, 不断扩展更有希望的邻居。它的优势来自图结构剪枝: 搜索只访问一小部分节点, 就能接近真实近邻集合。

问答 9: HNSW 的构图质量为什么重要

用户问题: HNSW 的构图为什么会显著影响查询性能?

AI 回答: HNSW 查询依赖图边能否把搜索引向正确区域。若邻接边过于局部, 搜索容易陷入局部区域; 若边过多, 访问和缓存开销又会上升。优秀的构图会保留具有导航价值的边, 同时删去冗余边, 使图既连通又具有较好的扩散性。课程提示中提到的三角不等式裁边, 本质上就是在邻居选择阶段降低冗余连接。

问答 10: 图索引查询内并行为什么困难

用户问题: 为什么 HNSW 查询内并行比 Flat 或 IVF 并行更难?

AI 回答: Flat 和 IVF 的候选集合相对明确, 可以把候选分片交给多个线程独立计算。HNSW 查询过程是动态的: 当前访问的节点会影响后续候选队列, 队列顺序又决定下一步扩展方向。若多个线程同时扩展不同节点, 可能产生重复访问、队列竞争和剪枝信息滞后。因此 HNSW 更适合 query-level 并行, 而查询内并行需要额外处理同步和搜索一致性。

问答 11: top-k 维护为什么会成为瓶颈

用户问题: 距离计算之外, top-k 维护为什么也会影响延迟?

AI 回答: ANN 查询最终要返回距离最小的一批候选。候选扫描越多, 系统越需要频繁比较、插入和淘汰元素。若每个线程维护局部 top-k, 最终还要归并多个局部结果; 若使用共享结构, 则会引入锁或原子同步。对 SIMD 和 GPU 来说, 距离计算可以高度并行, 但 top-k selection 往往包含分支、比较和不规则写入, 因此容易成为端到端瓶颈。

问答 12: 线程数增加为什么不一定带来线性加速

用户问题: 多线程 ANN 为什么经常达不到线性加速?

AI 回答: 线性加速要求工作量能均匀分配, 且同步、调度和内存带宽开销很小。ANN 中的候选长度、倒排链大小、图搜索路径都可能不均衡; 多个线程同时访问大数组或图邻接表时, 也会竞争 cache 和内存带宽。当单 query 工作量已经被索引结构大幅削减后, 线程创建、任务分发和结果归并的比例会变高, 继续增加线程数就可能收益递减。

问答 13: x86 与 ARM 的 ANN 性能差异通常来自哪里

用户问题: 同一套 ANN 算法在 x86 和 ARM 上为什么表现不同?

AI 回答: 差异主要来自 SIMD 宽度、单核频率、缓存层次、内存带宽、线程调度和编译器向量化能力。x86 平台常见 AVX2/AVX-512 能一次处理更多浮点或整数 lane; ARM/Kunpeng 使用 NEON 或 SVE 时, 向量宽度和指令吞吐不同。对于 HNSW 这类访存和分支较多的算法, 缓存命中率和线程运行时开销也会明显影响最终延迟。

问答 14: GPU 更适合哪类 ANN 工作负载

用户问题: ANN 中什么样的工作负载更适合 GPU?

AI 回答: GPU 适合大批量、规则、数据并行的工作负载, 例如批量 query 的候选扫描、矩阵式距离计算和结构化 top-k 选择。它不擅长大量细粒度分支、动态优先队列和随机图遍历。IVF batch scan 比

HNSW 直接图搜索更适合 GPU，因为倒排链扫描可以被组织成较规则的并行任务，而 HNSW 的搜索路径依赖 query 和运行时队列状态。

问答 15: OpenMP target 与普通 OpenMP 的概念差别是什么

用户问题： OpenMP target 和普通 OpenMP parallel 的区别是什么？

AI 回答： 普通 OpenMP parallel 主要在 CPU 线程之间分配工作，所有计算仍在主机端执行。OpenMP target 引入设备端执行模型，代码区域可以被编译并卸载到 GPU 或其他加速器。它涉及 host-device 数据映射、设备内核启动、teams/thread 层级和设备端内存访问。评估 OpenMP target 时，关键是确认计算确实发生在加速器上，并把数据传输、kernel 执行和同步开销都纳入分析。

问答 16: SYCL 与 OpenMP target 在表达模型上有什么差别

用户问题： oneAPI/SYCL 与 OpenMP target 都能使用 Intel GPU，它们的编程模型有什么不同？

AI 回答： OpenMP target 更接近指令式卸载，开发者通过 pragma 标注循环和数据映射；SYCL 是基于 C++ 的异构编程模型，显式使用 queue、buffer 或 USM、kernel lambda 和事件依赖来描述设备计算。OpenMP target 迁移 CPU 循环较直接，SYCL 对 kernel 组织、异步执行和跨设备抽象更细。二者性能差异取决于编译器优化、内存模型、kernel 粒度和数据移动方式。

问答 17: 构建时间和查询时间如何权衡

用户问题： ANN 报告为什么要同时讨论 build time 和 query latency？

AI 回答： 构建时间反映索引准备成本，查询延迟反映在线服务成本。Flat 几乎没有构建成本，但查询要扫描大量候选；HNSW 构建更慢，需要插入节点并维护图边，但查询速度很高；IVF 需要训练质心和建立倒排链，构建成本介于二者之间。若数据集静态且查询次数多，高构建成本可以被摊薄；若数据频繁更新，构建成本就会显著影响算法选择。

问答 18: 实验图表怎样表达科研论文风格

用户问题： ANN 性能图怎样才能更接近科研论文风格？

AI 回答： 科研图表应围绕明确假设或结论组织，而不是堆叠所有数据点。常见做法包括：用横轴表示 latency、纵轴表示 recall 或 speedup；用颜色区分平台或算法族；标出 $\text{recall}@100 \geq 0.95$ 约束线；对最优可行点加注释；将精确数值放入表格。图的目的是展示“跑了多少配置”，而是让读者快速看出参数变化、平台差异和算法 trade-off。

C 参考文献

参考文献

- [1] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE TPAMI*, 2020.
- [2] Z. Fu, W. Xiang, N. Wang, and D. Cai. Fast approximate nearest neighbor search with the navigating spreading-out graph. *PVLDB*, 2019.
- [3] H. Jegou, M. Douze, and C. Schmid. Product quantization for nearest neighbor search. *IEEE TPAMI*, 2011.
- [4] J. Johnson, M. Douze, and H. Jegou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.